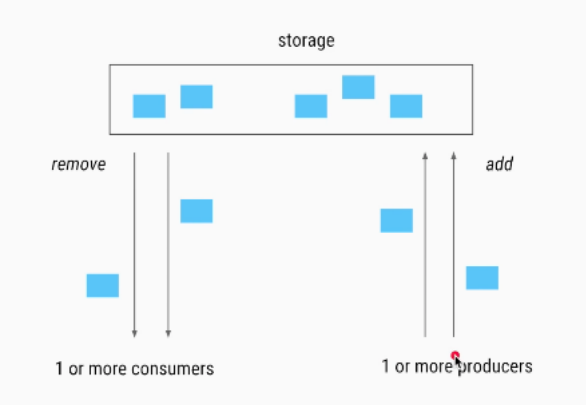
Producer-Consumer Pattern

19 January 2022 13:15

We have
- 1 or more producers, that produce items to storage
- 1 or more consumers, that consume these items from storage

Example of use: You're pushing to an API that has an upper ratelimit. You'll have to wait for a 200 response from the last push before you push the next one (CM pushing data to FDP via Dart)



- If there's no items in storage, consumer waits until producer has produced
- If there's no space in storage, producer waits until consumer has consumed

USE A BLOCKING QUEUE

BlockingQueue is constructed with a specified maximum size

Methods to enqueue

1. queue.put(element): Blocks thread until there's space in queue for element. Throws InterruptedException if the thread that is waiting to enqueue gets interrupted
2. queue.add(element): Will not block is there's no space, throws an IllegalStateExcpetion instead
3. queue.offer(element): Doesn't block, returns true or false (bool) if there wasn't space for the element
4. queue.offer(element, timeout): Doesn't block forever, only till the timeout. Also throws InterruptedException

Methods to dequeue

1. queue.take(): Blocks until element becomes available. Also throws InterruptedException if thread is interrupted.
2. queue.poll(): Return first element in the queue. Returns null if there's no element.
3. queue.poll(timeout): Waits for specified timeout for element to appear in queue. Throws InterruptedException.
4. queue.remove(element): Removes element if element is present in queue. If multiple elements match, it only removes the first el.

To remove all elements from queue to a container

1. queue.drainTo(collection)
2. queue.drainTo(collection, number of elements)

```
public static void main(String[] args) {

    BlockingQueue<Item> queue = new ArrayBlockingQueue<>(10);   <- Handles concurrent thread access

    // Producer
    final Runnable producer = () -> {
        while (true) {
            queue.put(createItem());
        }                                                       <- thread blocks if queue full
    };
    new Thread(producer).start();
    new Thread(producer).start();

    // Consumer
    final Runnable consumer = () -> {
        while (true) {
            Item i = queue.take();
            process(i);                                         <- thread blocks if queue empty
        }
    };
    new Thread(consumer).start();
    new Thread(consumer).start();

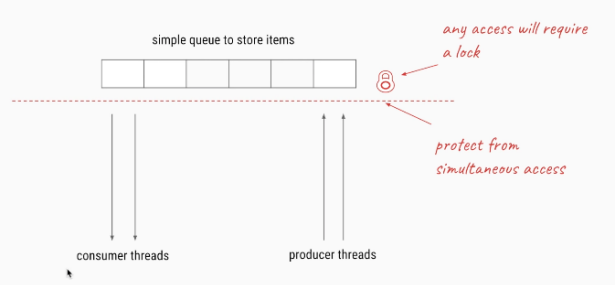
    Thread.sleep(1000);                                         try-catch for take and put skipped for brevity
}
```

DOING THIS WITHOUT BLOCKING QUEUE

We have to ways to do this
1. Using Locks and Conditions
2. Using Wait-Notify

USING LOCKS AND CONDITIONS

Basic DS to prevent concurrent access




```
public class MyBlockingQueue<E> {

    private Queue<E> queue;
    private int max = 16;
    private ReentrantLock lock = new ReentrantLock(true);

    public MyBlockingQueue(int size) {
        queue = new LinkedList<>();   <- can also use array, but need to track head and tail indexes
        this.max = size;
    }

    public void put(E e) {
        lock.lock();
        try {
            queue.add(e);
        } finally {                   <- protected by lock
            lock.unlock();
        }
    }

    public E take() {
        lock.lock();
        try {
            E item = queue.remove();
            return item;
        } finally {                   <- protected by same lock
            lock.unlock();
        }
    }
}
```

Block threads if queue is full when pushing, and when queue is empty when popping

```
public void put(E e) {
    lock.lock();
    try {
        if (queue.size() == max) {
            // block the thread       <- block until queue has atleast 1 empty slot to add item
        }
        queue.add(e);
    } finally {
        lock.unlock();
    }
}

public E take() {
    lock.lock();
    try {
        if (queue.size() == 0) {
            // block the thread       <- block until queue has atleast 1 item to take
        }
        E item = queue.remove();
        return item;
    } finally {
        lock.unlock();
    }
}
```

USING WAIT-NOTIFY

```
private Object notEmpty = new Object();
private Object notFull = new Object();

public synchronized E take() {
    if (queue.size() == 0) {
        notEmpty.wait();
    }
    E item = queue.remove();
    notFull.notifyAll();
    return item;
}
```

```
private Object notEmpty = new Object();
private Object notFull = new Object();

public synchronized void put(E e) {
    while (queue.size() == max) {
        notFull.wait();
    }
    queue.add(e);
    notEmpty.notifyAll();
}
```

Block threads if queue is full when pushing, and when queue is empty when popping

```
private int max;
private Queue<E> queue = new LinkedList<>();
private ReentrantLock lock = new ReentrantLock(true);
private Condition notEmpty = lock.newCondition();
private Condition notFull = lock.newCondition();

public MyBlockingQueue(int size) {
    queue = new LinkedList<>();
    this.max = size;
}

public void put(E e) {
    lock.lock();
    try {
        if (queue.size() == max) {
            notFull.wait();
        }
        queue.add(e);
        notEmpty.signalAll();
    } finally {
        lock.unlock();
    }
}

public E take() {
    lock.lock();
    try {
        if (queue.size() == 0) {
            notEmpty.wait();
        }
        E item = queue.remove();
        notFull.signalAll();
        return item;
    } finally {
        lock.unlock();
    }
}
```